

Object-oriented scientific programming with C++

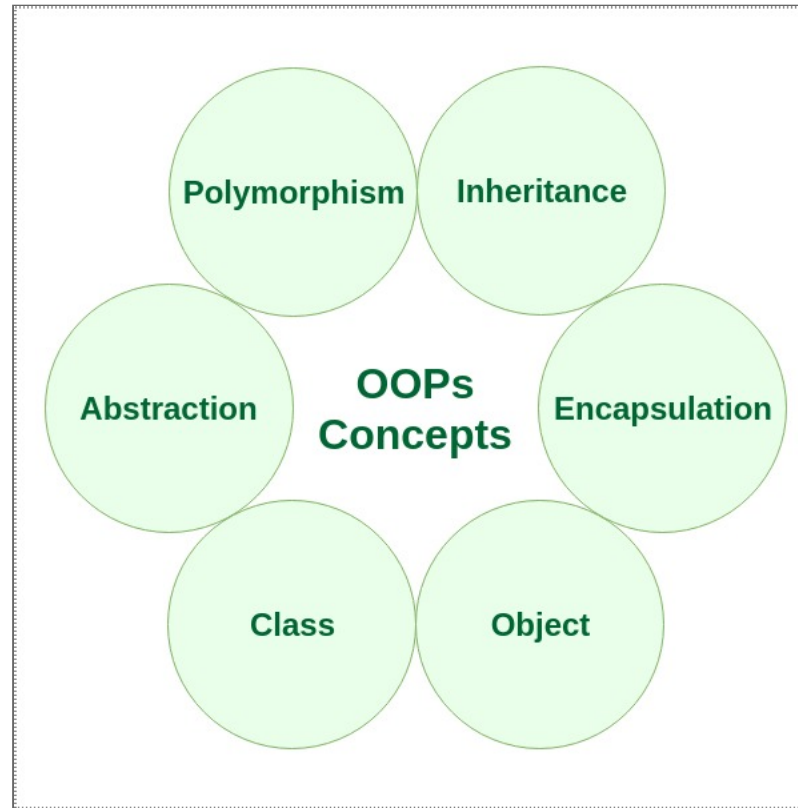
Matthias Möller, Jonas Thies, Călin Georgescu, Jingya
Li (Numerical Analysis, DIAM)
Lecture 1

What's this course about?

1. Principles of object oriented programming (not restricted to C++)
2. Principles of scientific programming (also not restricted to C++)
3. C++ 11, 14, 17, 20 and some of 23, not C++ 03 and before

Object oriented programming

Using LEGO blocks as an analogy, you can think of object-oriented programming (OOP) as a way of building complex structures (programs) by piecing together different types of blocks (objects). These are the main concepts of OOP



A first example of OOP concepts

Matlab

Python

Here, `size` is a standalone **function** that is applied to the matrix A from outside. That means that `size` must be able to deduce the matrix size. In other words, the matrix size is **publicly visible**.

Here, matrix A provides a **member attribute** to report its size from inside. The attribute is also **publicly visible** but offers more fine-grained control.

Course information

- Lectures (nonobligatory)
 - weeks 2.1-2.8, Wed 13:45-15:45 in lecture hall Ampere
- Lab sessions (nonobligatory)
 - weeks 2.2-2.8, Tue 8:45-12:45 (IDE Ctrl+Enter)
 - this is the time and place to ask your questions (no office hours! no reply to emails! no reply to discussion forums!)

- WebLab (<https://weblab.tudelft.nl/tw3720tu-wi4771tu/2025-2026>)
 - all demos, self-study exercises, computer exam and the final projects are provided via WebLab
 - weekly demos and self-study exercises become available on Monday morning
 - final projects become available in the week before Christmas and must be submitted by the end of Q2 via WebLab

- Assessment (3 ECTS)
 - weekly self-study exercises (formative feedback, no grade)
 - 3h individual closed-book computer exam on 16 December 2025 (40% of the grade)
 - final project can be worked on in groups of 1-3 students (60% of the grade)

- Evaluation: self-study exercises
 - programming exercises are checked against full unit tests; you can submit at any time and as often as you wish and receive immediate feedback
 - open questions are checked by TAs after the submission deadline; you get feedback within one week

- Evaluation: computer exam
 - programming assignments are checked against full unit tests; you can submit as often as you wish and receive immediate feedback; the grade is calculated by the percentage of passed unit tests
 - open questions are checked by examiners based on answer models after the exam

- Evaluation: final project
 - programming assignments are checked against preliminary unit tests during the exam; you can submit as often as you wish and receive immediate feedback if your implementation meets the specifications (*check that your code is callable*)
 - programming assignments are checked against full unit tests after the exam; your grade is determined by the percentage of passed unit tests (*check that your code functions correctly*)
 - open questions are checked by examiners based on answer models after the deadline

- Unit tests
 - don't try to reverse engineer the unit tests!
Focus on writing correct code instead
 - don't ask us to write the unit tests so that they tell you which line of your code needs to be changed and how!
 - write your code according to all requirements of the assignment, especially adhere to the given interfaces
 - test your code carefully and think about corner cases, e.g., math-operations between different data types
 - if you cannot find the error ask us during the lab sessions

- Fraud

- It is prohibited to distribute the material in full or in parts in any form (printed and electronically). This, in particular, prohibits the transfer of the material to webservices like Bitbucket, GitHub, Gitlab, etc. and the making available of solutions to the final project. This action is considered piggybacking and will be treated as fraud.
- It is not forbidden to use ChatGPT or Co-Pilot as a source of inspiration. However, you must understand the code you submit and you must be able to explain your code when asking for help from TAs. Questions like *"This is what ChatGPT came up with, can you*

make it pass the unit tests?" will not be answered.

- The computer exam will be in closed-book style, i.e. no internet connection

After all formalities ...

- ... ENJOY the course, I did so in all the years
- ... LEARN to write good C++ code
- ... DARE to think out-of-the-box
- ... ASK questions, I and my TAs are happy to answer them

Let's get started with the fun part

Programming languages

- A computer programming language is a formal notation (set of instructions) for writing computer programs
- One distinguishes between
 - Interpreted languages like Python, JavaScript, ...
 - Compiled languages like C, C++, Fortran, Julia, ...
- Another distinction is between
 - Functional programming that treats computation as evaluation of math functions
 - Object-oriented programming that treats computation as living objects that have

mutable data and provide methods to
manipulate the data

Pros and Cons of *interpreted* languages

Python: Hello.py

- Processes the code line by line at run time (slow!)
 - ▯
 - Computes `len(a)` twice (in lines 3, 5) ▯
 - Cannot know in advance that lines 5-6 are not 'used' at all ▯
 - Cannot reorder code ▯

- Can inject/load new code instructions at run time □
- Platform independent □
- Dynamic or no typing □ / □

Pros and Cons of *compiled* languages

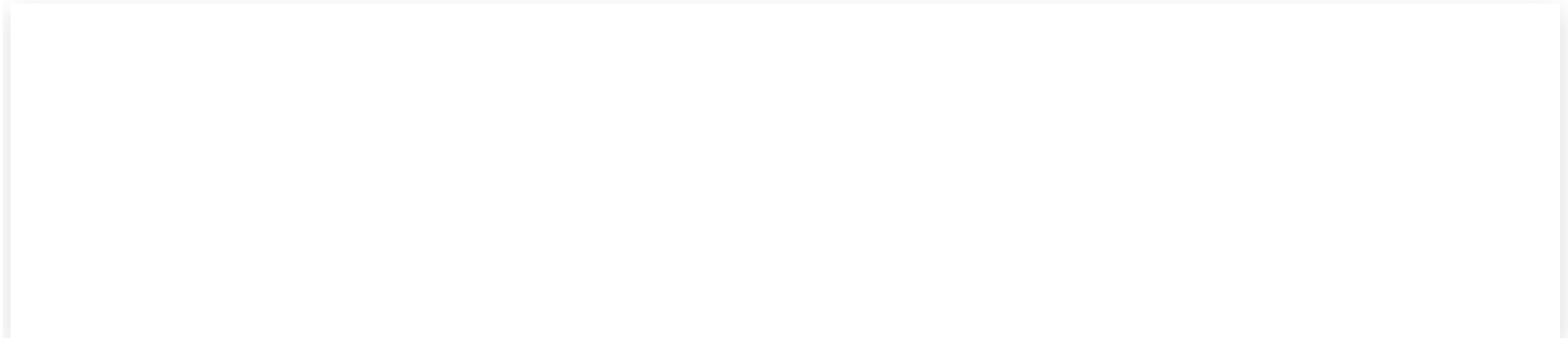
C++: Hello.cxx

- Pre-processes the code line by line at compile time to create (optimized) run time executable
 - Evaluates lines 2-3 at compile time and replaces and by values □
 - Eliminates 'dead code' such as lines 1-7 □
- (Very) platform dependent □

- Compiler can check types □

Example of a *compiled* language

C++: Hello.cxx



Compilation by hand

yields the following files

- contains (later!)
- is the assembly code (text)
- is the object file (binary)
- is the executable (binary)

Running the executable

outputs

Let's get started

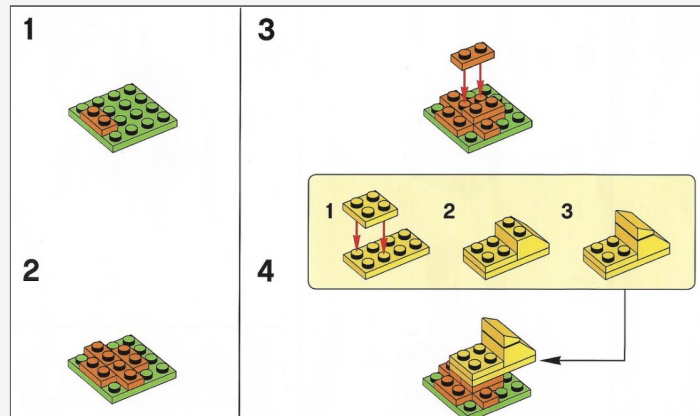
- Live demo in WebLab:
<https://weblab.tudelft.nl/tw3720tu-wi4771tu/2025-2026/>
- Where to find information on C++
 - A Tour of C++: <https://isocpp.org/tour>
 - CPlusPlus: <http://www.cplusplus.com>
 - Geeks for Geeks:
<https://www.geeksforgeeks.org/c-plus-plus/>

Class vs. Object

Once again in LEGO terms ...

A **class** is a **blueprint**, aka an instruction manual that tells you how to create something

An **object** is a living instance created (instantiated) from the class blueprint



VS.



Let's start with the following example, and learn everything (first two weeks) by filling in this example

A large, empty rectangular box with a thin gray border, intended for the user to fill in the example.

Hello World!



This can also be done by using a class



Extra functionality provided by the standard C++ library is defined in so-called header files which need to be included via

Some useful header files are

- `<iostream>` : input/output
- `<string>` : string types
- `<complex>` : complex numbers
- Good overview: **<http://www.cplusplus.com>**

We will write our own header files later in this course as well

OOP style and member function

Function that computes the sum of a Vector

This is NOT really OOP-style!

The member function version

This is a GOOD OOP-style!

We will get back to this topic later

THE main function

Now let's only focus on the function

- Each C++ program must provide one (and only one!) global main function which is the designated start of the program

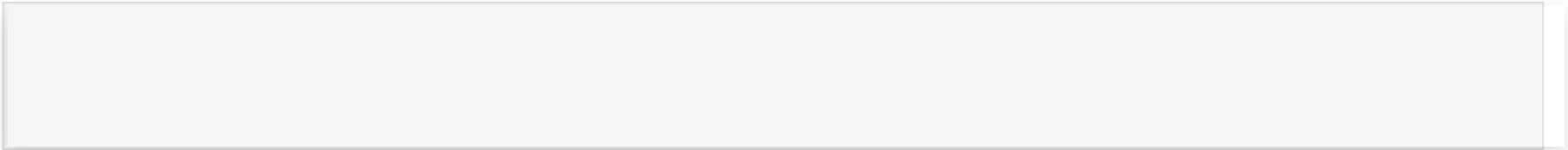
or

- Scope of the main function is
- Return type of the main function is (=integer), e.g.,
- Main function cannot be called recursively

Standard output

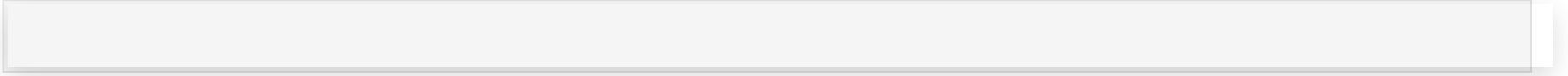
Stream-based output system

In [1]:



Streams can be easily concatenated

In [2]:



Streams are part of the standard C++ library and therefore encapsulated in the namespace `std`; instead of using `std::` one can also import all functionality from the namespace by

In [3]:

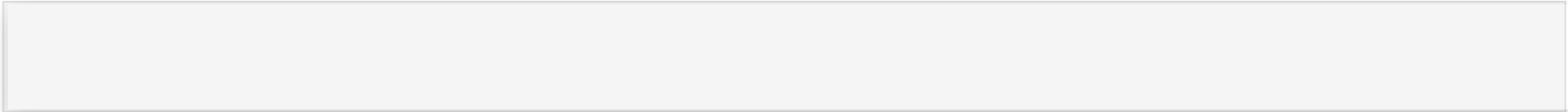
Predefined output streams

- `System.out` : standard output stream
- `System.err` : standard output stream for errors
- `System.log` : standard output stream for logging

Variables and constants

C++ is case sensitive and typed, that is, variables and constants have a value and a concrete type

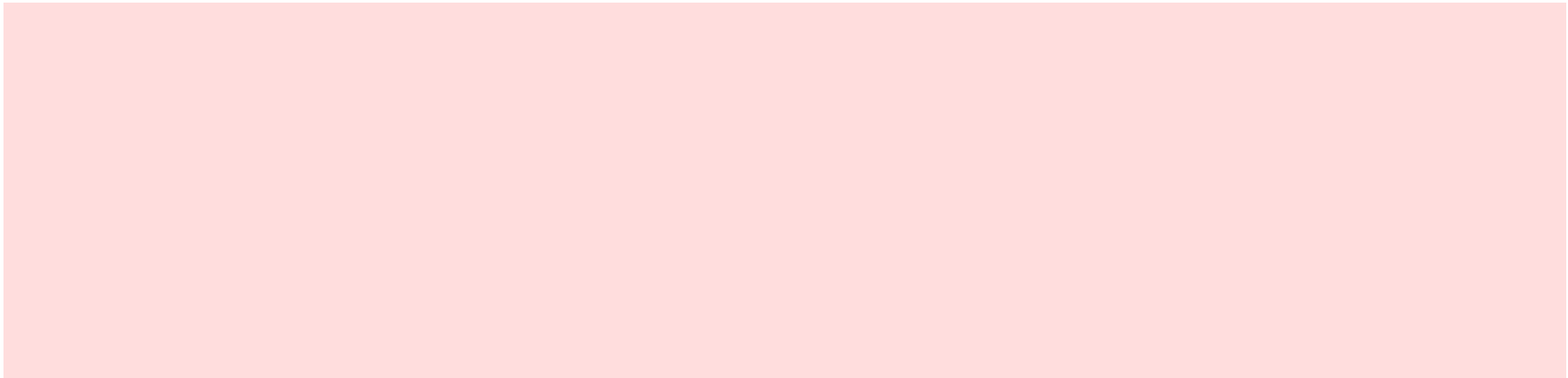
In [4]:



Out[4]:

Variables can be updated, constants cannot

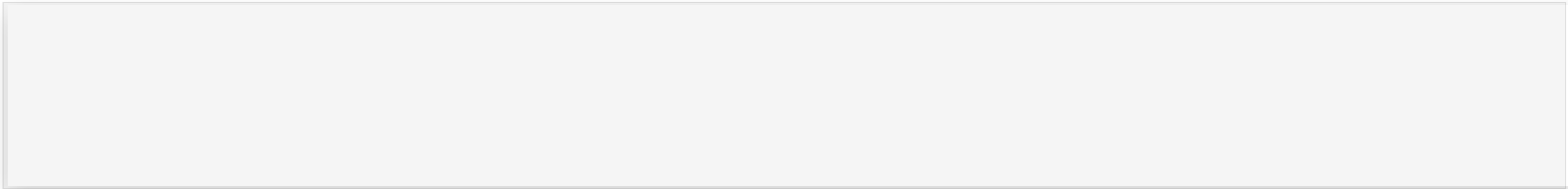
In [5]:



Initialization of constants

Constants must be initialized during their definition

In [6]:



Initialization of variables

Variables can be initialized during their definition or (since they are variable) at any location later in the code

In [7]:

Out[7]:

Intermezzo: Terminology

- A declaration introduces the name of the variable or constant, and describes its type but does not create it

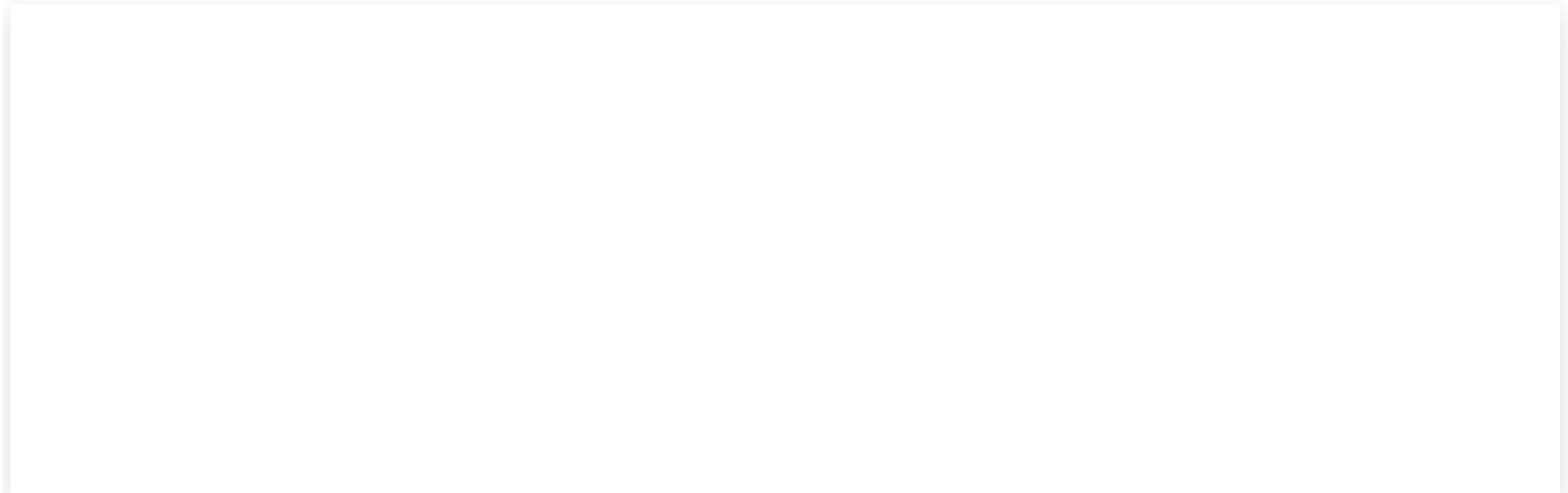
- A definition instantiates it (=creates it)

- An initialization initializes it (=assigns a value to it)

- All three steps can be combined, e.g., `int x = 1;` or split across different source/header files (later in this course)

Scope of variables/constants

Variables/constants are only visible in their scope



Another example



C++ standard types

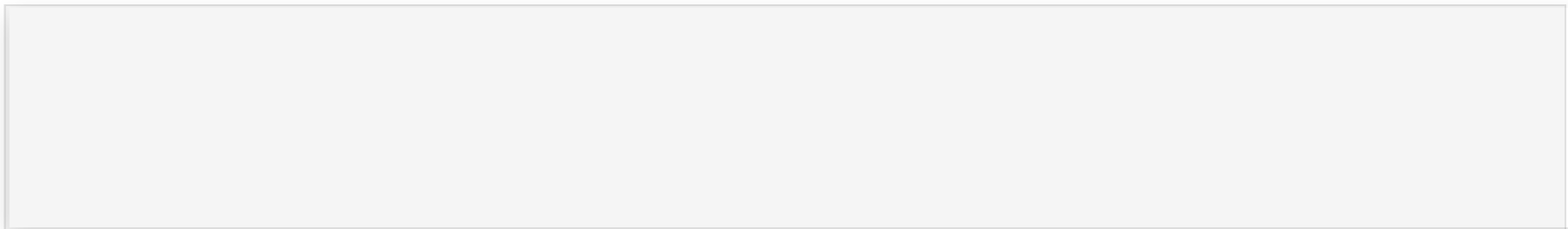
Group	Type name	Notes on size / precision
Character types		Exactly one byte in size. At least 8 bits.
		Not smaller than char. At least 16 bits.
		Not smaller than char16_t. At least 32 bits.
Integer types (signed and unsigned)		Same size as char. At least 8 bits.
		Not smaller than char. At least 16 bits.

Group	Type name	Notes on size / precision
		Not smaller than short. At least 16 bits.
		Not smaller than int. At least 32 bits.
		Not smaller than long. At least 64 bits.
Floating-point type		Precision not less than float
		Precision not less than float
		Precision not less than double
Boolean type		

Examples of C++ types

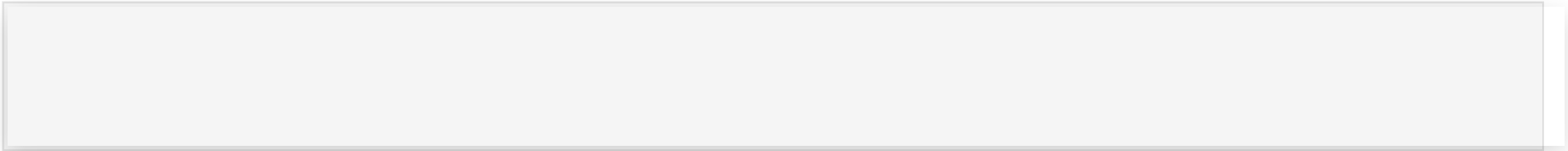
Double-precision floating-point type

In [8]:



Single-precision floating-point type

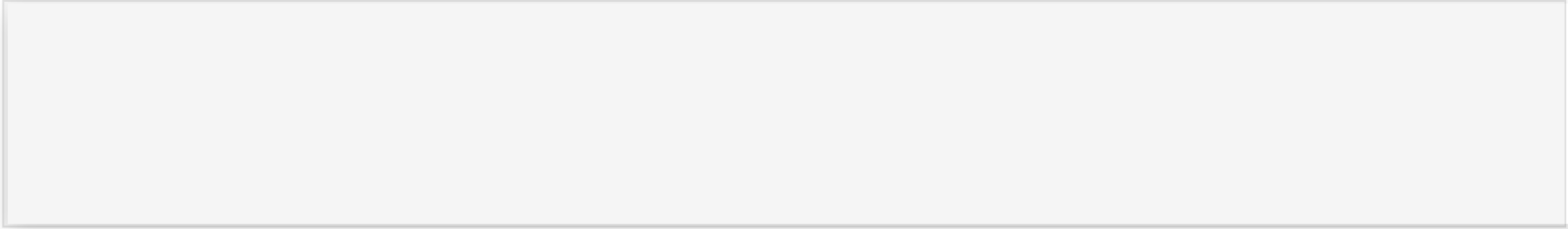
In [9]:



Mixing and conversion of types

Getting the type of a variable

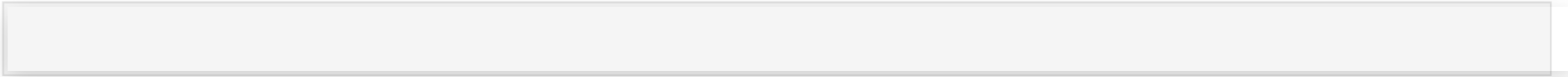
In [10]:



Out[10]:

C++ converts different types automatically

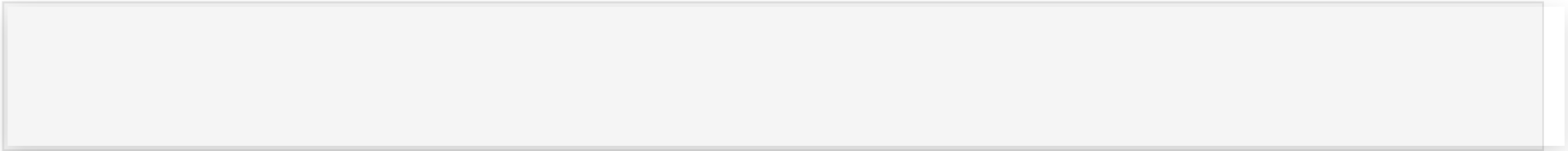
In [11]:



Out[11]:

Check if you are happy with the result

In [12]:



Out[12]:

C++11 introduces the `constexpr` keyword, which makes handling of mixed types very easy

In [13]:

Out[13]:

You can also explicitly cast one type into another

In [14]:

Out[14]:

auto vs. explicit types

- Recommendation: use the keyword
 - to improve readability of the source code
 - to improve maintainability of the source code
 - to benefit from performance gains (later in this course)
- ... unless explicit conversion is required
 - `auto` is the same as `decltype(auto)`
 - `decltype(auto)` is the same as `decltype(auto)`
- ... unless the C++ standard does not allow so, e.g., return type of a (pure) virtual function (later in this course)

Use of suffix type specifiers

- Suffix type specifiers (termed literals) seem unnecessary at first glance since constants are implicitly converted

- But keep in mind that `int` and `long` are not the same

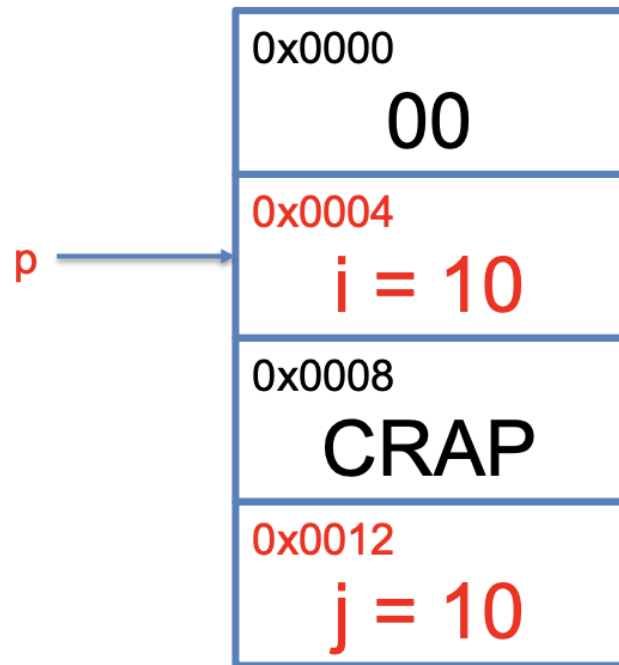
In [15]:

Address-of/dereference operators

- Integer variable

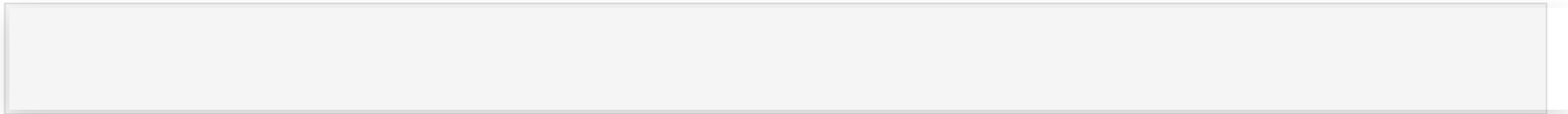
- Pointer to its address

- Dereference to its value



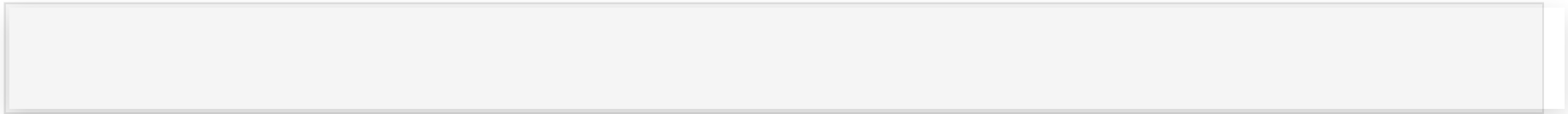
Address-of operator (`&`): returns the address of a variable (= its physical location in the computer's main memory)

In [34]:



Addresses are of pointer type (equal to that of the variable)

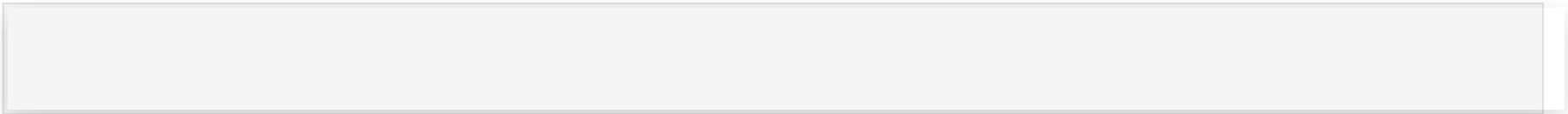
In [35]:



Out[35]:

Dereference operator (`*`): returns the value behind the pointer (= the value stored at the physical location in the computer's main memory)

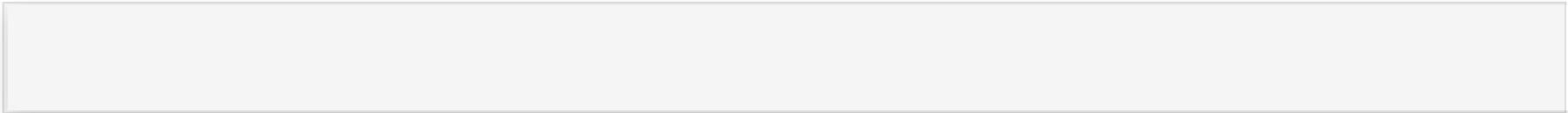
In [36]:



Pointers and references

Pointers can be used to have multiple variables (with different names) pointing to the same value, i.e. the same location in the computer's main memory

In [37]:



Let us change the value of variable

In [38]:

Out[38]:

Dereference pointer and change its value

In [39]:

Out[39]:

Change value of pointer without dereferencing it

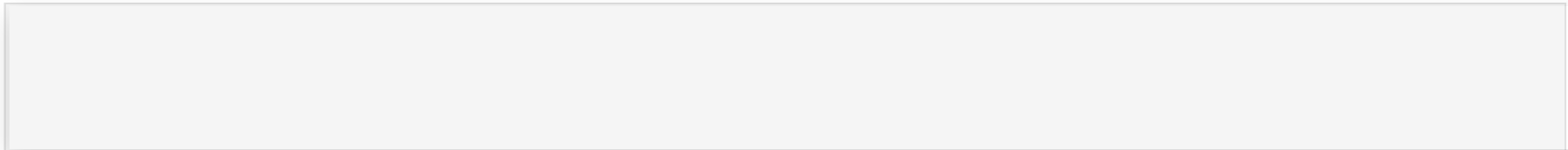
In [40]:

Out[40]:

Pointer hazards

Pointers that remain uninitialized can cause hazard

In []:



C++11 introduces the new keyword `nullptr` that explicitly sets a pointer to null

In [42]:

You can use `if (ptr != nullptr)` to check if a pointer can be dereferenced without problems

Error handling with exceptions

Let us include the `sys` header file

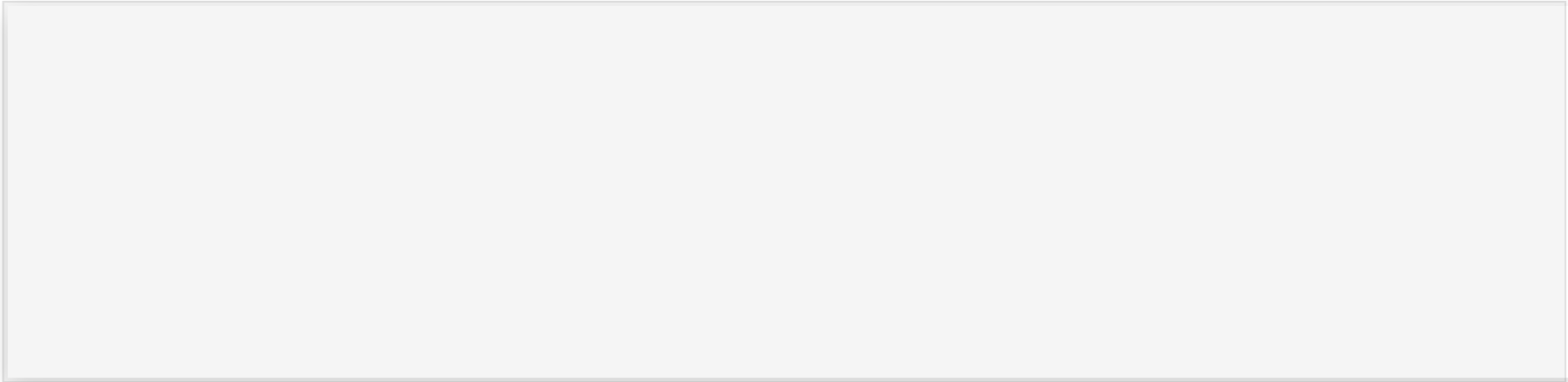
In [44]:

Use `signal` to signal the occurrence of an anomalous situation

In [45]:

Use `try` and `finally` blocks to handle exceptions gracefully

In [46]:



Error handling with assertion

Let us include the `<assert.h>` header file and assert that a condition is true

In [48]:

Quiz: Why does nothing happen here?

Let us turn on debug mode by explicitly disabling the non-debug () mode

In []:

The function evaluates the inside parentheses. If it evaluates to , will print an error message and then terminate the program but only if the code is compiled in debug mode.
DON'T USE IT IN WEBLAB

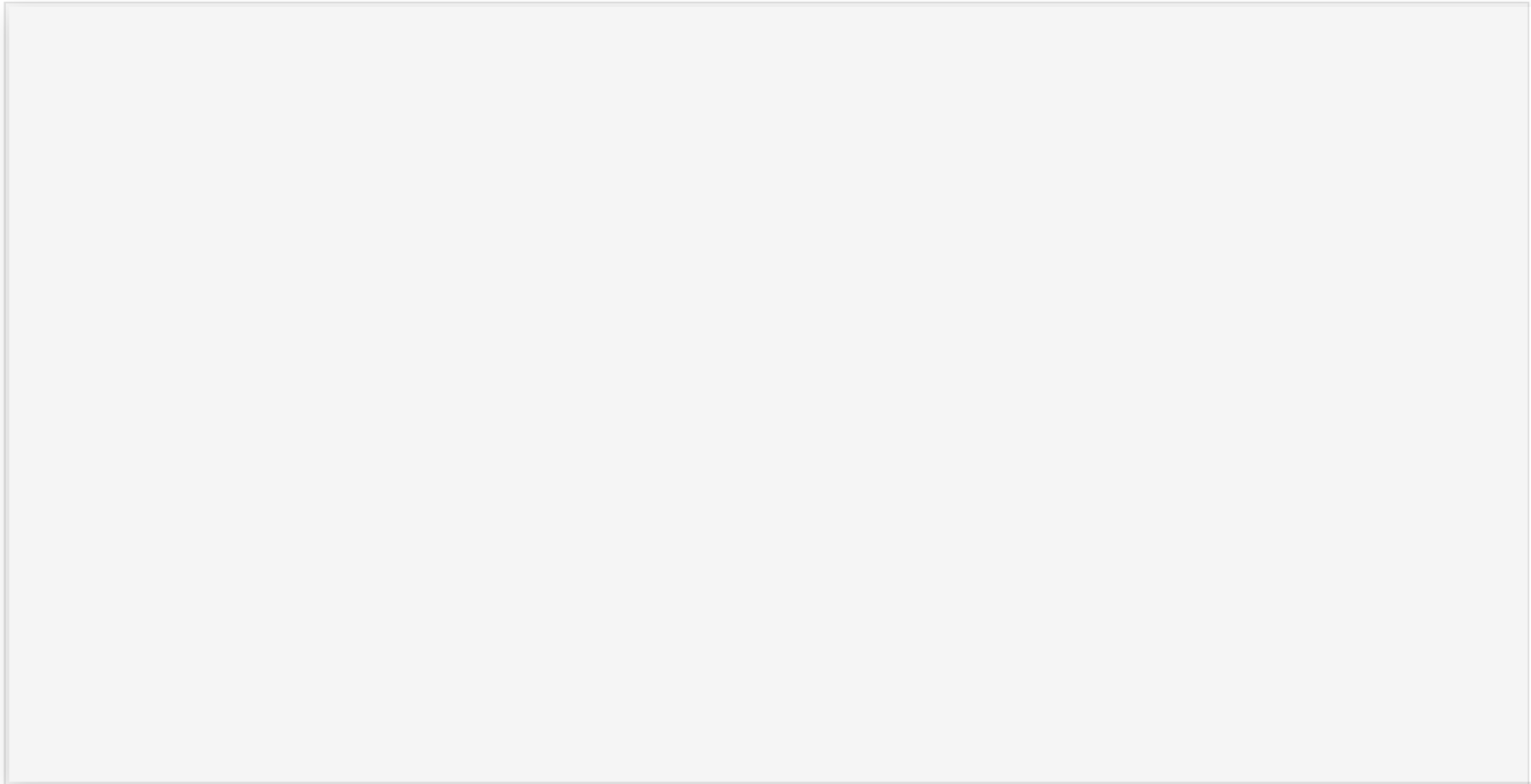
Error handling best practices

What are the best practices of error handling?

- Prefer exceptions for signaling errors over return codes.
- Only use exceptions for exceptional conditions, not normal flow control.
- Ensure all exceptions are caught and handled appropriately.

Debug example: divide by zero

In []:



Argument passing – by value

Arguments that are passed by value are passed as a physical duplicate of the original variable

In [2]:

Quiz: What happens if variable is not of type but a vector of several gigabyte?

Argument passing – by reference

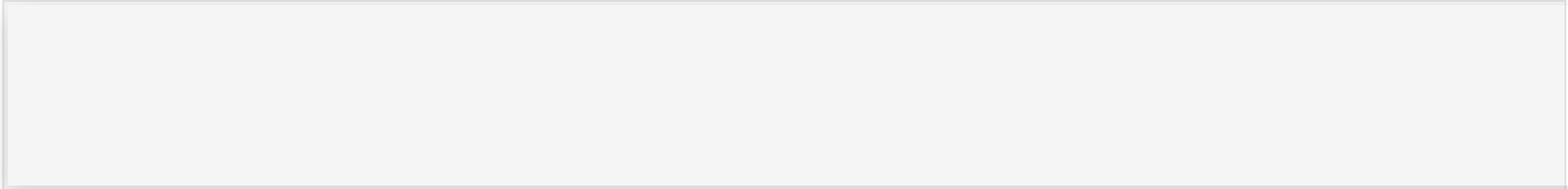
Arguments that are passed by reference give the function read and write access to the memory location of the original variable

In [3]:

Quiz: What happens if `myFunction` tries to modify the value of the passed variable?

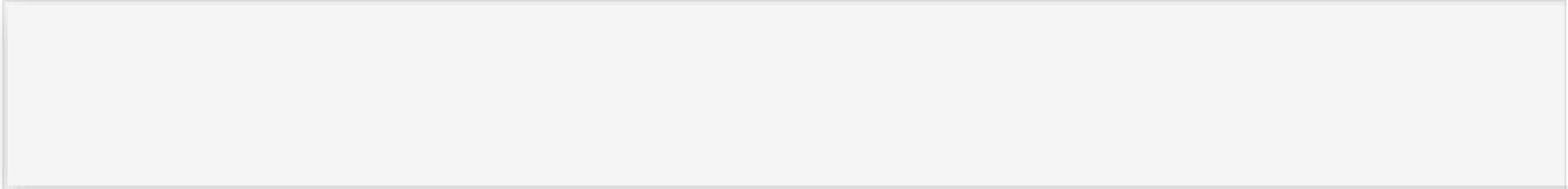
The post-increment operator performs the operation first and increments variable afterwards

In [4]:



The pre-increment operator increments variable first and performs the operation afterwards

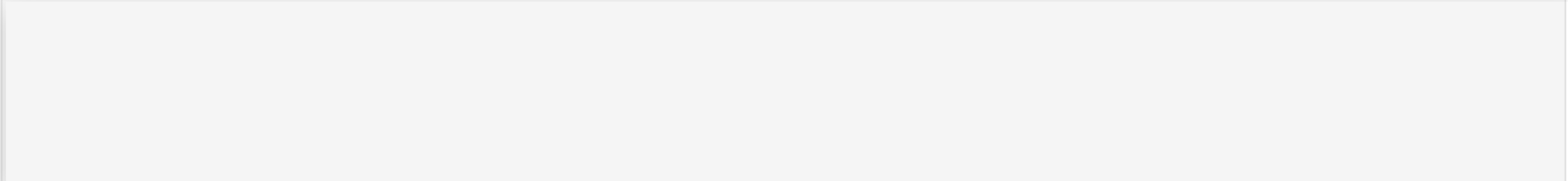
In [5]:



Argument passing – by constant reference

Arguments that are passed by constant reference give the function read access to the memory location of the original variable

In [6]:



C++ return value optimization (RVO)

Most C++ compilers support RVO, that is, no temporary variable for the return value is created inside the function

but the return value is immediately assigned to variable

Argument passing

If we want a function that changes the argument directly, we must pass the argument by reference.

The return type `void` indicates that 'nothing' is returned.

Argument passing – by address

Passing by address

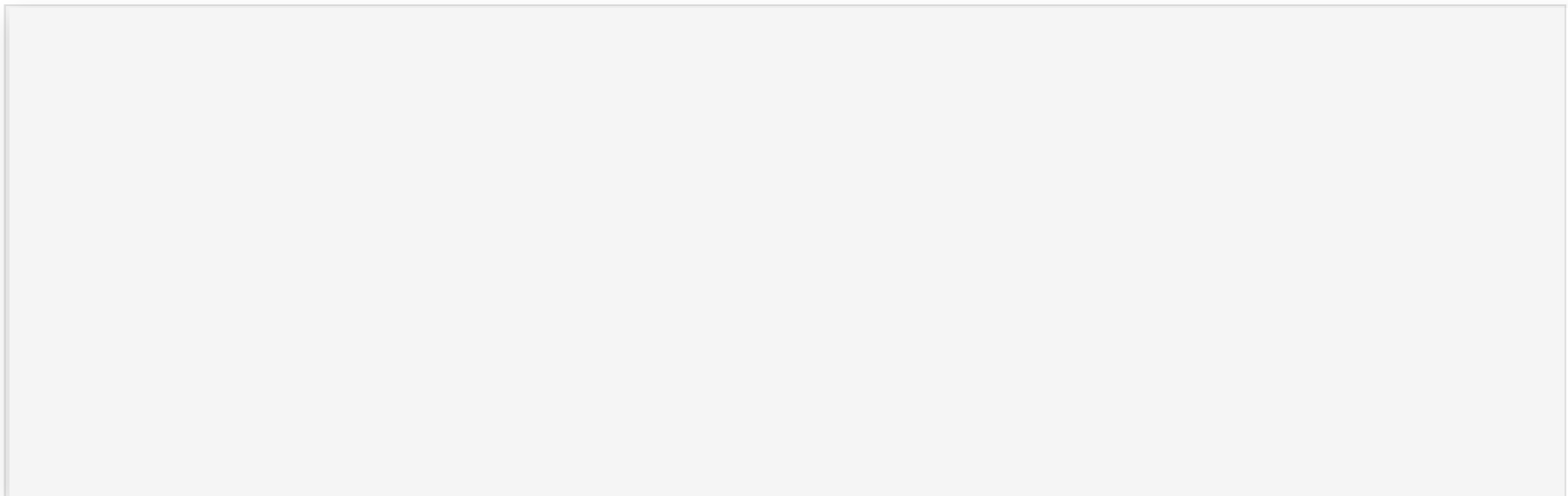
In [7]:

This is the old C-style to pass arguments that should be modifyable inside the function or to pass arrays, aka the first position in the computer's main memory at which the array starts.

Example

Compute the sum of the entries of an array

In [9]:



This is not OOP. DON'T, REALLY DON'T DO THIS IN C++!
We will learn much better ways to pass bigger objects

such as arrays by (constant) reference.

There is one exception. If you want to allocate memory dynamically inside the function and assign it to a variable defined outside the function you need to work with double pointers.

Static arrays

Definition and creation of a static array

Definition, creation and initialization of a static array

Access of individual array positions

In [10]:

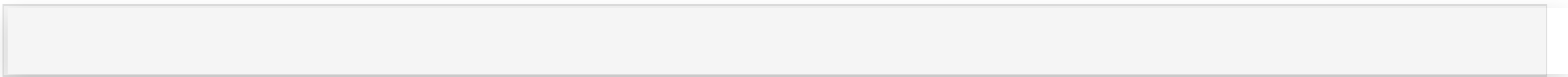
Remember that C++ starts indexing at 0

Static arrays are destroyed automatically at the end of scope

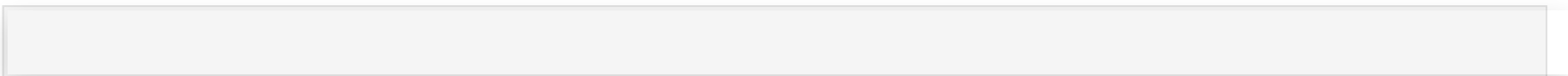
Quiz: Static arrays

What happens?

In []:

A horizontal rectangular box with a light gray background and a thin gray border, intended for entering code.

In []:

A horizontal rectangular box with a light gray background and a thin gray border, intended for entering code.

Quiz: `char* argv[]`

What is `char* argv[]`?

Example use case:

Dynamic arrays

- Definition and allocation of dynamic array

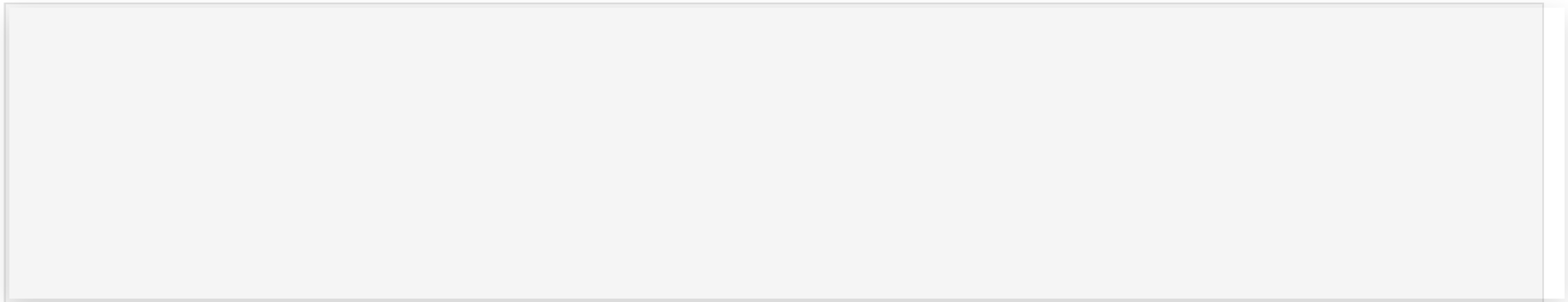
- Definition, allocation and initialization of dynamic array

- Explicit deallocation of dynamically allocated array needed

- Think fail-safe! Because it still points to an invalid address

Example of a dynamic array

In [11]:

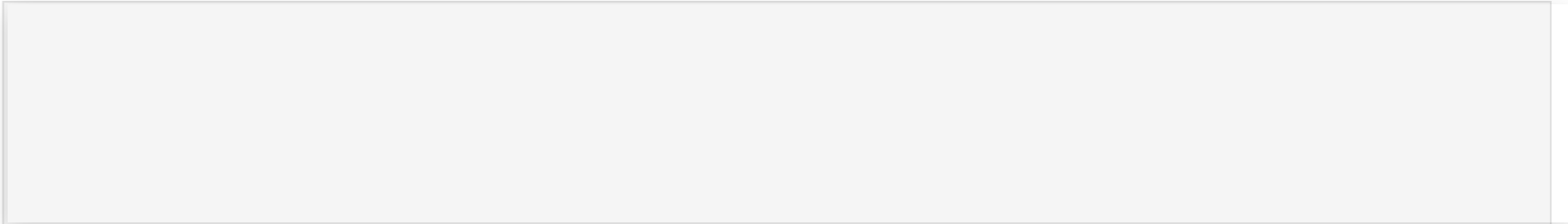


1	2	3	4	5
---	---	---	---	---

Static vs. dynamic arrays

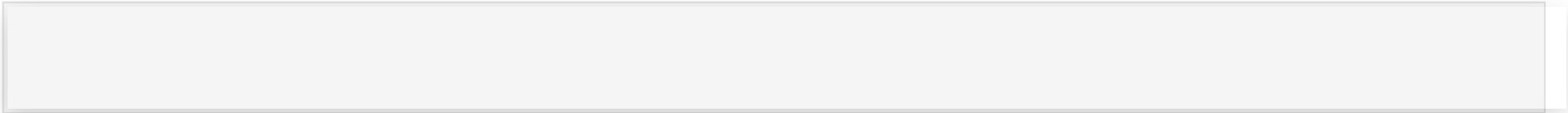
Static arrays require the size to be known at compile time

In []:

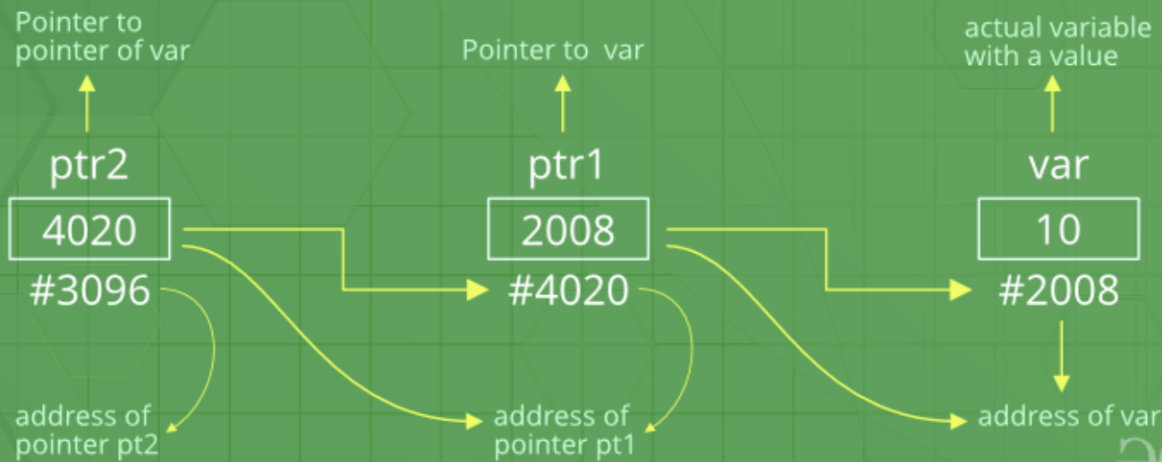


Dynamic arrays allow variable sizes at run-time

In []:



Double Pointer



Namespaces

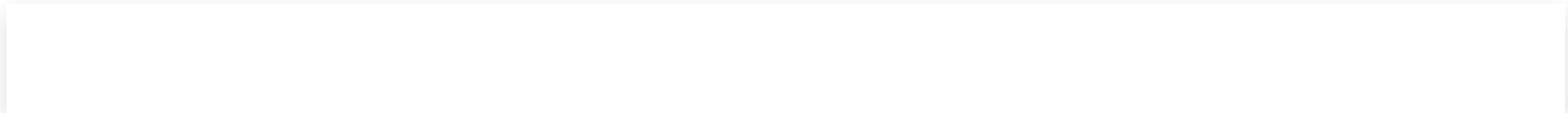
Namespaces, like `from module import *`, allow to bundle functions even with the same function name (and interface) into logical units

In []:

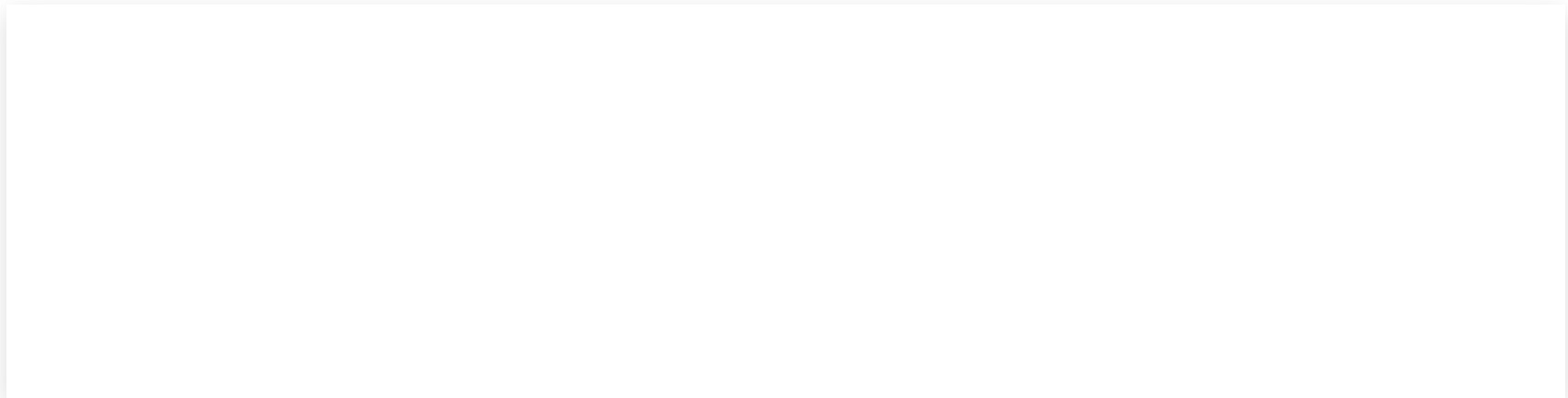
In []:

Functions implemented in a namespace can be called by

- providing the namespace explicitly

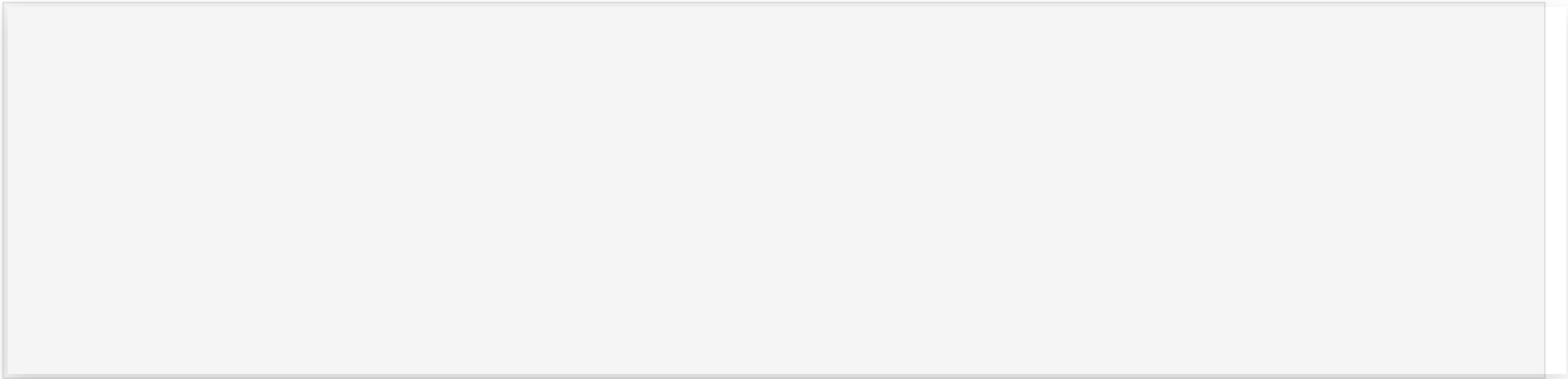


- importing the namespace into the scope



Namespaces can be nested

In []:



Leading `::` goes back to outermost unnamed namespace

In []:

